

Session 1: AI for Coding – Tools, Concepts, and Setup

Elira Kuka & Tanner Regan

The George Washington University

What We Will Cover (Sessions 1 & 2)

- ▶ Part 1: Tools, concepts, and setup
 - Overview of AI coding tools and how to choose among them
 - How to use AI Agents: Terminal, Desktop App, VS Code
 - What Git is and why it matters for research
 - Python coding with AI

What We Will Cover (Sessions 1 & 2)

► Part 1: Tools, concepts, and setup

- Overview of AI coding tools and how to choose among them
- How to use AI Agents: Terminal, Desktop App, VS Code
- What Git is and why it matters for research
- Python coding with AI

► Part 2: Practical workflows

- Writing, debugging, and automating code with AI
- “Vibe coding” from plain English descriptions
- Project organization and replication packages
- Demo: data cleaning script from scratch

Coding Perhaps Highest-Value Use of AI

- ▶ AI most reliable when output is verifiable: code runs or it does not
- ▶ Economists spend enormous time on repetitive coding tasks
 - AI can handle most of them
- ▶ Massive productivity gains across skill levels:
 - Beginners: “vibe code” to write code in an unfamiliar language (no syntax!)
 - Advanced users: automate drudge work, prototype faster, reduce bugs

Coding Perhaps Highest-Value Use of AI

- ▶ AI most reliable when output is verifiable: code runs or it does not
 - ▶ Economists spend enormous time on repetitive coding tasks
 - AI can handle most of them
 - ▶ Massive productivity gains across skill levels:
 - Beginners: “vibe code” to write code in an unfamiliar language (no syntax!)
 - Advanced users: automate drudge work, prototype faster, reduce bugs
- ⇒ First two sessions will cover how to use AI to complete coding tasks

Overview of AI Coding Tools

The AI Coding Landscape: Many Options

- ▶ General-purpose AI assistants (good at coding, among many other things):
 - **Claude** (Anthropic) — our focus today
 - **ChatGPT / GPT-4o** (OpenAI)
 - **Gemini** (Google)
- ▶ Agentic coding tools (read files, write and run code autonomously):
 - **Claude Code** (Anthropic) — our focus today
 - **Codex** (OpenAI) — closest competitor to Claude Code
 - **GitHub Copilot** (Microsoft) — inline suggestions in editor
 - **Cursor / Windsurf** — AI-first code editors
- ▶ How to choose: task type, privacy needs, and workflow integration

Chatbot vs. Agentic Tool

- ▶ **Chatbot** (Claude.ai, ChatGPT): you ask → it answers → *you* act
- ▶ **Agentic tool** (Claude Code): you give a goal → it reads files, writes code, runs it, fixes errors, iterates → reports back
- ▶ For economists: instead of “here is some code,” it runs the code and tells you what it found

Which One Should You Use? Claude Code vs. Codex

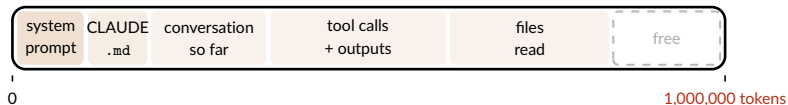
- ▶ Two serious options for agentic coding today: **Claude Code** and **Codex**
- ▶ Plans mirror each other at each price point:
 - \$20/mo: Claude Pro vs. ChatGPT Plus
 - \$100/mo: Claude Max (5x) vs. ChatGPT Pro (5x)
 - \$200/mo: Claude Max (20x) vs. ChatGPT Pro (20x)
- ▶ Both use a 5-hour rolling window + weekly cap; limits reset automatically
- ▶ At the same price, Codex currently gives more usage per dollar

Which One Should You Use? Claude Code vs. Codex

- ▶ Two serious options for agentic coding today: **Claude Code** and **Codex**
- ▶ **Codex** has some advantages:
 - More usage per subscription dollar — important for heavy users
 - Sometimes stronger on pure coding benchmarks
- ▶ **Claude Code** has some advantages:
 - Often better at tasks beyond code: understanding context, writing, reasoning
- ▶ **Bottom line:** this comparison is fast-moving and will keep changing
 - I am using Claude Code and find it excellent for my workflow
 - I would switch only if Codex proved clearly and consistently superior
 - My advice: pick one, learn it well, reassess in 6 months

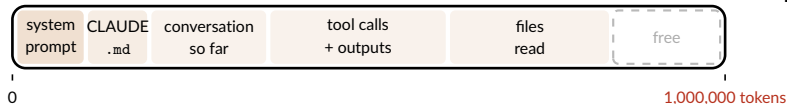
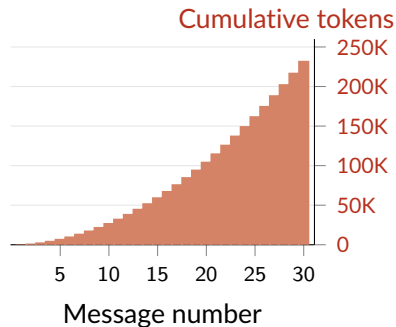
What You Pay For: Tokens and Context

- ▶ **Token:** the unit AI reads and writes — roughly $\frac{3}{4}$ of a word
 - **You pay for:** tokens *in* (prompt + files) and tokens *out* (reply) — input is cheaper
- ▶ **Context window:** everything the model holds at once — fills as session grows



What You Pay For: Tokens and Context

- ▶ **Token:** the unit AI reads and writes — roughly $\frac{3}{4}$ of a word
 - **You pay for:** tokens *in* (prompt + files) and tokens *out* (reply) — input is cheaper
- ▶ **Context window:** everything the model holds at once — fills as session grows



Getting the Most Out of Your Subscription

- ▶ Subscriptions have rolling limits — fewer tokens per task means more use
- ▶ How to use fewer tokens:
 - Start a new session for each new task: context accumulates and raises costs
 - Store standing instructions once so you never repeat yourself
 - Be specific: vague prompts waste tokens and invite unfocused replies
 - Ask multiple things at once so less back and forth
 - Compact your chats/context window
- ▶ Silly trick: start Claude when wake up so 5-hour window ends earlier in day

Claude Code / Codex Interfaces

Three Ways to Use an Agentic Coding Tool

▶ **Terminal** (command line):

- Most powerful; works directly in your project directory
- Best for: running scripts, data analysis, replication tasks

▶ **Desktop App:**

- Friendlier interface; easier to get started
- Projects feature: persistent context across sessions
- Best for: drafting, quick questions, early exploration

▶ **VS Code integration:**

- Agent lives inside your editor — see code and AI side by side
- Best for: sustained development work, Git integration, Jupyter/Python

Terminal vs. VS Code: Quick Comparison

Terminal (CLI)

- ▶ Lightweight; fast startup
- ▶ Works anywhere, incl. remote servers
- ▶ Keyboard-driven; fast for quick tasks
- ▶ No inline diffs; steeper setup

VS Code Extension

- ▶ Inline diffs; accept changes block-by-block
- ▶ Code & AI panel side by side
- ▶ Best for large codebases, sustained work
- ▶ Heavier on CPU and memory

Live Demo: Terminal

- ▶ ▷ *Demo slide – minimal text, walk through live*
- ▶ Navigate to a project directory
- ▶ Ask Claude Code to describe what it sees
- ▶ Ask it to write a simple data task

Live Demo: Desktop App

- ▶ ▷ *Demo slide – minimal text, walk through live*
- ▶ Create a Project and load context
- ▶ Same task as terminal – compare experience

Live Demo: VS Code

- ▶ ▷ *Demo slide – walk through VS Code interface live*
- ▶ Same task as terminal – compare experience

Contex Engineering with Claude Overview

Context Engineering

▶ Context Window

- ▶ Each prompt to Claude Code bundles the entire history of your session
 - ▶ your prompts, Claude's responses, files it read, code outputs, system prompts
- ▶ These are all appended until the context window hits its fixed size limit
 - ▶ For Claude Code this is around 200k tokens
- ▶ Claude then “**auto-compacts**” writes its own summary and then restarts fresh
 - ▶ less preferable to frequent intentional compaction

▶ Context Engineering

- ▶ You can tell Claude when to compact using `/compact your instructions`
- ▶ Develop habits that the context window focussed on relevant information
 - ▶ Claude.md vs skills —and— ‘ask before edits’ vs ‘plan’ Modes

Read more at [Paul GP's Guide](#)

Anatomy of a context window (example from Paul GP's Guide)

```
<|start|>system<|message|>You are ChatGPT, a large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-06-28  
Reasoning: high  
# Valid channels: analysis, commentary, final.  
Channel must be included for every message.  
Calls to these tools must go to the commentary channel: 'functions'.<|end|>
```

system prompt
(written by openAI)

ANATOMY OF A
MODERN LLM
CHAT

```
<|start|>developer<|message|># Instructions  
Be concise and technical. Provide code examples when relevant.
```

developer
message
(tool definitions)

```
# Tools  
## functions  
namespace functions {  
  type web_search = (_: {  
    query: string,  
    num_results?: number,  
  }) => any;  
} // namespace functions<|end|>
```

user
prompt

```
<|start|>user<|message|>What's new in the latest version of polars?<|end|>
```

thinking

```
<|start|>assistant<|channel|>analysis<|message|>User asking about latest polars version.  
My knowledge cutoff is 2024-06, need to search for recent updates.<|end|>
```

```
<|start|>assistant<|channel|>commentary<|message|><|call|>functions.web_search  
{  
  "query": "polars python latest version new features", "  
  "num_results": 5  
}<|end|>
```

tool call

```
<|start|>tool<|message|>[{"title": "Polars 1.8.0 Release Notes", "snippet": "New  
streaming engine, improved join performance, added pivot_wider...", "url": "..."}]<|end|>
```

tool
results

```
<|start|>assistant<|channel|>final<|message|>Polars 1.8.0 (latest as of Nov 2025)  
includes:
```

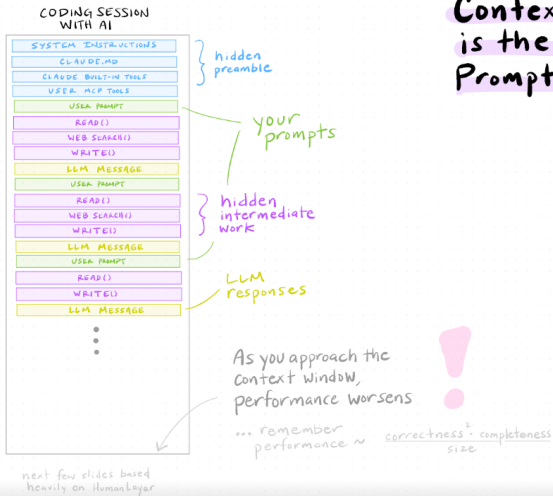
- **Streaming engine overhaul**: Better memory efficiency for large datasets
- **Join performance**: 2-3x faster for certain join types
- **New API**: `pivot_wider()` for reshaping data
- **LazyFrame improvements**: Enhanced query optimization

final response

```
Check the release notes for migration details if upgrading from older versions.<|return|>
```

You
typically
see only
these
two!

Context engineering (example from Paul GP's Guide)



Context Engineering
is the new
Prompt Engineering

Context auto-compact (example from Paul GP's Guide)

If you hit the
content window
the LLM will
auto-compact



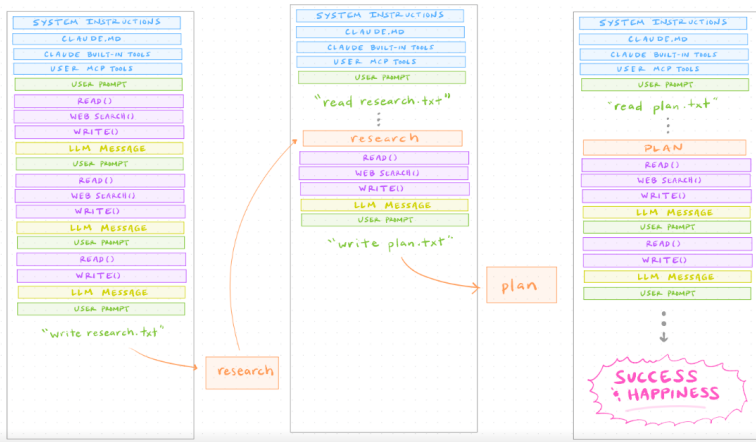
the model summarizes your conversation

...this is
rarely
ideal...



Context intentional compacting (example from Paul GP's Guide)

Frequent intentional compaction is better



CLAUDE.md and Skills

- ▶ **CLAUDE.md** (memory) — Claude reads at the start of every session
 - Store context that is always true so you don't repeat yourself in every prompt
 - `.claude/CLAUDE.md` is global, `your-project/CLAUDE.md` is *project* specific
 - Ask Claude to update after substantial changes to project
- ▶ **Skills** — Claude reads every time it is called
 - Store context for a task that you repeat often, but not always relevant
 - `.claude/skills/your-skill/SKILL.md` is called with `/your-skill`
 - Write and update your own skills or download from others

Read more at [Claes Backman's Guide](#) (full descriptions, examples of CLAUDE.md and skills, download pre-made skills)

your-project/CLAUDE.md (example from Claes Backman's Guide)

```
# Project: [Paper title]

## Data
- Unit of analysis: firm-year panel, 2010-2022
- Raw data lives in data/raw/ – never edit these files
- Main dataset after cleaning: data/clean/panel.dta (N ≈ 47,000)

## Code conventions
- R scripts are numbered (01_clean.R, 02_analysis.R, ...)
- All regressions cluster SEs at the firm level
- Use fixest for all panel regressions

## LaTeX
- Main file: paper/main.tex
- Tables generated by 03_tables.R go to paper/tables/
- Use \widehat{} not \hat{} for estimators
```

`.claude/skills/robustness/SKILL.md` (example from Claes Backman's Guide)

```
---  
name: robustness  
description: Run standard robustness checks on the main regression.  
---
```

For the main specification in this project:

1. Re-run with alternative clustering (industry-year instead of firm)
2. Re-run dropping the top and bottom 1% of the outcome variable
3. Re-run on the pre-2020 subsample only
4. Produce a summary table comparing coefficients across all specs

Claude Modes

▶ Ask before edits

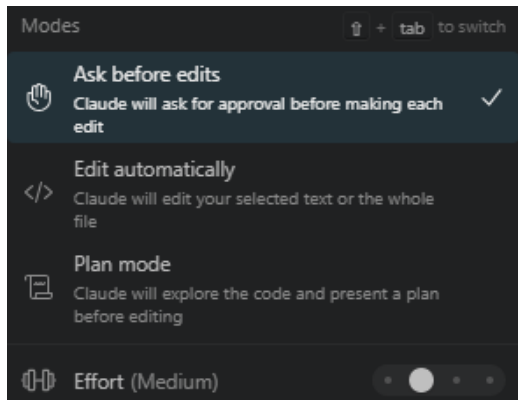
- ▶ Do steps one at a time
- ▶ Asks your permission before running each and every command

▶ Edit automatically

- ▶ Same as above but runs commands without asking permission (risky)

▶ Plan

- ▶ Claude writes plan.md with many steps
- ▶ You review and execute entire plan



VS Code with Claude Overview

What is VS Code?

- ▶ Integrated Development Environment (IDE)
 - ▶ Free, open source (created by Microsoft), and very popular
- ▶ Core features:
 - **Explorer:** file browser and project tree
 - **Editor:** syntax highlighting for every language (Stata, R, Python, $\text{\LaTeX}$, ...)
 - **Integrated terminal:** run scripts without leaving the editor
 - **Extensions:** Git, Jupyter, Claude Code, and more
 - **Command Palette** (`Ctrl+Shift+P`): search and run any action by name
- ▶ Resources: [Official intro videos \(Microsoft\)](#), [Tips and Tricks \(Microsoft\)](#)

<https://paulgp.substack.com/p/getting-started-with-claude-code>

VS Code with Claude Overview

- ▶ Key panels: Explorer (files), Editor, Terminal, Source Control (Git), Extensions, Claude Code
- ▶ CC Modes:
 - **Ask before edits:** Claude proposes each change; you approve
 - **Plan mode:** Claude writes a full plan in Markdown before doing anything
 - **Auto:** Claude acts autonomously (use carefully)

Extensions Reading and rendering files Claude extension: Current file selection

Git: Version Control for Economists

Why Git Matters for Research

- ▶ Version control = a complete history of every change to your code
- ▶ No more: `analysis_v3_FINAL_revised2.do`
- ▶ Enables:
 - Rolling back to any prior version of your code
 - Working safely with coauthors and RAs without overwriting each other
 - Tracking exactly what changed between draft versions
 - Automatic replication audit trail
- ▶ The good news: Claude Code can handle all Git commands for you — you just describe what you want

Key Git Concepts (No Commands Required)

- ▶ **Repository (repo):** a folder whose history Git is tracking
- ▶ **Commit:** a snapshot of your code at a point in time, with a message
- ▶ **Push / Pull:** sync your local repo with GitHub (the remote copy)
- ▶ **Branch:** a parallel version of the code for a new feature or robustness check
- ▶ **Merge:** fold a branch back into the main code
- ▶ Typical workflow for an economist:
 1. Work on code; when something works, commit it
 2. Push to GitHub at end of day (backup + coauthor access)
 3. RA works on a branch; you review and merge

Git and Claude Code: What the Integration Looks Like

- ▶ ▷ *Placeholder: screenshot or demo*
- ▶ In VS Code: Git panel shows all changes; Claude Code can stage, commit, push on request
- ▶ You tell Claude: “commit my changes with a descriptive message” — it writes the message too
- ▶ Branching for research: create a branch for each robustness check, merge if it works
- ▶ Connecting with Overleaf:
 - Overleaf Pro can sync with a GitHub repo
 - Alternatively: LaTeX Workshop extension in VS Code + local compilation

Python in VS Code: Environments

- ▶ ▷ *Demo slide – Tanner*
- ▶ Anaconda manages Python installations and packages
- ▶ Each project gets its own **environment**: a fixed set of packages and versions
 - Prevents conflicts between projects
 - Ensures others can reproduce your results exactly
- ▶ Ask Claude Code to create and activate an environment for you

Running Python Code with Jupyter

- ▶ ▷ *Demo slide – Tanner*
- ▶ Three modes in VS Code:
 - **Jupyter notebook** (`.ipynb`): interactive cells, inline output – good for exploration
 - **Interactive window**: run `.py` scripts cell-by-cell – best of both worlds
 - **Script** (`.py`): run the full pipeline – best for production code
- ▶ Claude Code can write, run, and debug Python in any of these modes

Setting Up Your Environment

What You Need to Install

- ▶ Core tools (all free except AI subscription):
 1. **VS Code** — the editor (free)
 2. **Claude Code** — VS Code extension + CLI (requires subscription)
 3. **Git** — version control (free); create a GitHub account
 4. **Python** — Anaconda distribution recommended (free)
- ▶ VS Code extensions to install:
 - Claude Code, GitLens, Jupyter, Python, Stata Enhanced (if applicable)
- ▶ We have a step-by-step setup guide — see handout / shared folder

Session 1 — Key Takeaways

1. AI has perhaps most value added in coding
2. Agentic $\neq$ chatbot: reads files, writes and runs code, fixes errors, reports back
3. Two serious agentic options today: Claude Code and Codex
 - Pick one, learn it, reassess in 6 months
4. Tokens accumulate fast in long session — start new session for each new task
5. Three interfaces (Terminal, Desktop, VS Code) and Git are your essential stack

Questions?

- ▶ Questions on tools, concepts, or setup?
- ▶ Thoughts on how this fits your current workflow?